

WEEK-12

Task-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
the_boss@DESKTOP-I8C8AJG:~/skill$ ls -l | grep .c | sort | head
-rw-r--r-- 1 the_boss the_boss 328 Sep 27 11:19 memory_test.c
-rw-r--r-- 1 the_boss the_boss 357 Sep 27 11:12 error_test.c
-rw-r--r-- 1 the_boss the_boss 425 Sep 27 10:34 foreground.c
-rw-r--r-- 1 the_boss the_boss 539 Sep 18 14:34 input_redirect.c
-rw-r--r-- 1 the_boss the_boss 542 Sep 27 11:03 process_group.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 18 14:39 output_redirect.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 27 10:38 signals.c
-rw-r--r-- 1 the_boss the_boss 596 Sep 18 14:43 append_redirect.c
-rw-r--r-- 1 the_boss the_boss 599 Sep 27 10:30 jobtable.c
-rw-r--r-- 1 the_boss the_boss 616 Sep 18 14:47 stderr_redirect.c
the_boss@DESKTOP-I8C8AJG:~/skill$ cat jobtable.txt | grep Running | sort
cat: jobtable.txt: No such file or directory
the_boss@DESKTOP-I8C8AJG:~/skill$ sleep 20 &
sleep 30 &
sleep 40 &
[1] 3201
[2] 3207
[3] 3213
the_boss@DESKTOP-I8C8AJG:~/skill$ jobs
[1]  Running                sleep 20 &
[2]-  Running                sleep 30 &
[3]+  Running                sleep 40 &
the_boss@DESKTOP-I8C8AJG:~/skill$ time ls -l | grep .c | sort | head
-rw-r--r-- 1 the_boss the_boss 328 Sep 27 11:19 memory_test.c
-rw-r--r-- 1 the_boss the_boss 357 Sep 27 11:12 error_test.c
-rw-r--r-- 1 the_boss the_boss 425 Sep 27 10:34 foreground.c
-rw-r--r-- 1 the_boss the_boss 539 Sep 18 14:34 input_redirect.c
-rw-r--r-- 1 the_boss the_boss 542 Sep 27 11:03 process_group.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 18 14:39 output_redirect.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 27 10:38 signals.c
-rw-r--r-- 1 the_boss the_boss 596 Sep 18 14:43 append_redirect.c
-rw-r--r-- 1 the_boss the_boss 599 Sep 27 10:30 jobtable.c
-rw-r--r-- 1 the_boss the_boss 616 Sep 18 14:47 stderr_redirect.c

real    0m0.019s
user    0m0.003s
sys      0m0.035s
[1]  Done                sleep 20
[2]-  Done                sleep 30
```

Report: Large pipelines and multiple background jobs were executed to test the stability of the system.

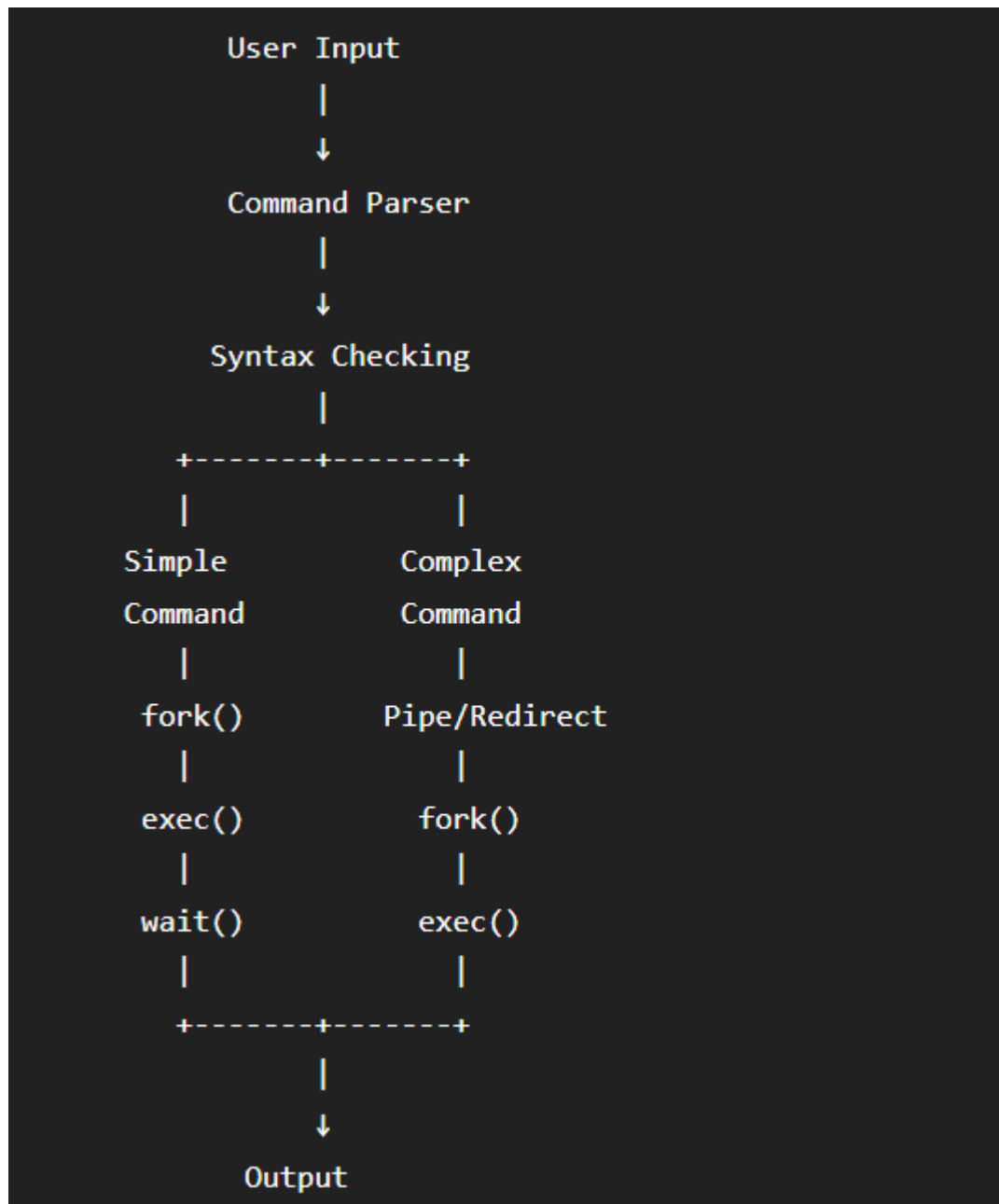
CPU and memory usage were monitored using Linux process-monitoring commands.

Failed commands were also tested to check whether errors were handled without crashing the program.

Execution time was measured to study the performance of pipelines and process management.

Task-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

Architecture:



Important system calls

System Call	Purpose
fork()	Creates a child process
exec()	Executes a command
wait()/waitpid()	Waits for child processes
pipe()	Creates communication channel
dup2()	Performs I/O redirection
open()	Opens files
read()	Reads data
write()	Writes data
close()	Closes file descriptors
kill()	Sends signals

Report: The architecture and execution flow of the project were documented using diagrams and descriptions.

The major system calls used for process creation, pipes, redirection, and signals were explained.

Important design decisions and the interaction between different modules were recorded.

The documentation was reviewed to provide a clear understanding of the complete project.

Task-3: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
GNU nano 8.7.1                                README.md
# Linux Shell Project

## Description
A simple Linux shell implementing process creation,
pipes, I/O redirection, background jobs, signals,
and command execution.

## Features
- Command execution
- Input/output redirection
- Pipes
- Background jobs
- Signal handling
- Command history
- Error handling

## Compilation

gcc *.c -o myshell

## Execution

./myshell

## Testing

Run the test scripts from the tests directory.

## System Calls Used

fork()
exec()
pipe()
dup2()
wait()
open()
read()

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Report: The project source code and supporting files were organized into a clear repository structure.

A README file was added to explain the project, features, compilation, and execution steps. The repository was checked to ensure that the required source files and documentation were present.

The final deliverables were reviewed and a version tag was created for the completed project.

Task-4: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
the_boss@DESKTOP-I8C8AJG:~/skill$ ls -l | grep .c | sort | head
-rw-r--r-- 1 the_boss the_boss 328 Sep 27 11:19 memory_test.c
-rw-r--r-- 1 the_boss the_boss 357 Sep 27 11:12 error_test.c
-rw-r--r-- 1 the_boss the_boss 425 Sep 27 10:34 foreground.c
-rw-r--r-- 1 the_boss the_boss 539 Sep 18 14:34 input_redirect.c
-rw-r--r-- 1 the_boss the_boss 542 Sep 27 11:03 process_group.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 18 14:39 output_redirect.c
-rw-r--r-- 1 the_boss the_boss 593 Sep 27 10:38 signals.c
-rw-r--r-- 1 the_boss the_boss 596 Sep 18 14:43 append_redirect.c
-rw-r--r-- 1 the_boss the_boss 599 Sep 27 10:30 jobtable.c
-rw-r--r-- 1 the_boss the_boss 616 Sep 18 14:47 stderr_redirect.c
the_boss@DESKTOP-I8C8AJG:~/skill$ sleep 20 &
[1] 3393
the_boss@DESKTOP-I8C8AJG:~/skill$ jobs
[1]+  Running                  sleep 20 &
the_boss@DESKTOP-I8C8AJG:~/skill$ |
```

```
the_boss@DESKTOP-I8C8AJG:~/skill$ sleep 20 &
[1] 3393
the_boss@DESKTOP-I8C8AJG:~/skill$ jobs
[1]+  Running                  sleep 20 &
the_boss@DESKTOP-I8C8AJG:~/skill$ ./signals
PID: 3414
Press Ctrl+C or Ctrl+Z to test signals.
Running...
Running...
Running...
Running...
Running...
Running...
Running...
^C
SIGINT received
Running...
Running...
Running...
Running...
```

```
Running...
Running...
Running...
Running...
Running...
Running...
Running...
User defined signal 1      ./signals
[1]+  Done                  sleep 20
the_boss@DESKTOP-I8C8AJG:~/skill$
the_boss@DESKTOP-I8C8AJG:~/skill$
```

Report: Final demonstration scenarios were prepared to verify the major features of the project. Pipelines, background jobs, signals, and process groups were demonstrated during the review. The complete project was checked for functionality, documentation, source organization, and reliability.

The final testing confirmed that the implemented components were ready for presentation and submission.